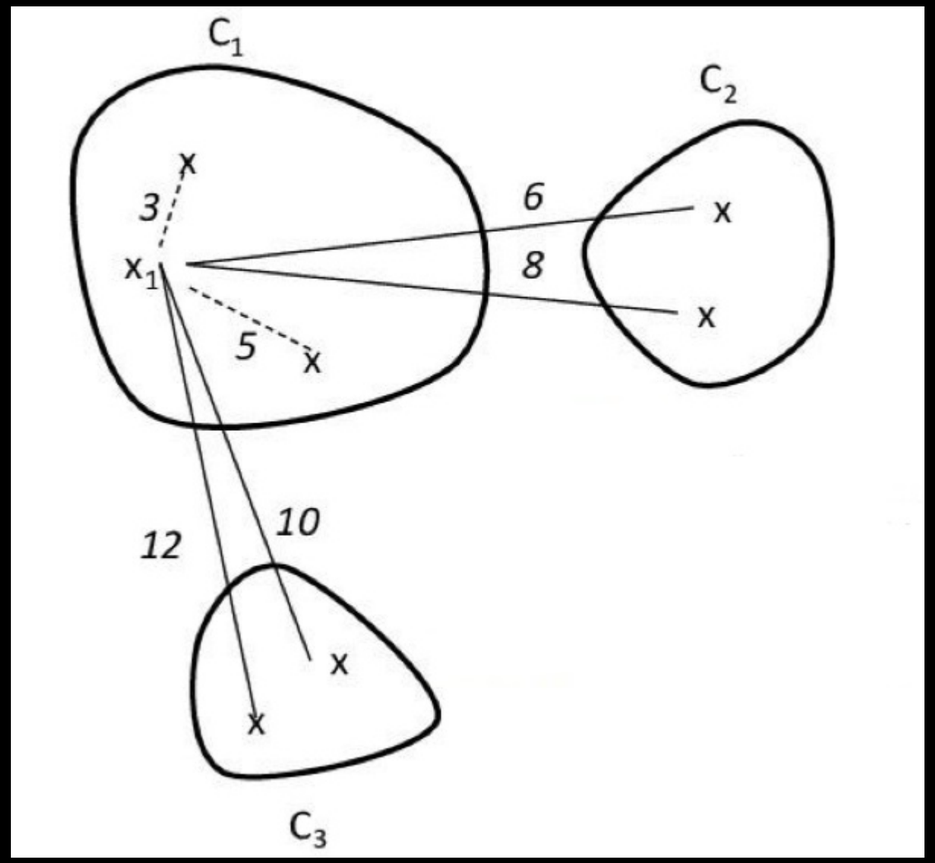
Agenda

- → Silhouette score
- → Kmeans from Scratch
- → Problem with K-means
- → K-means ++

Silhoutte Score



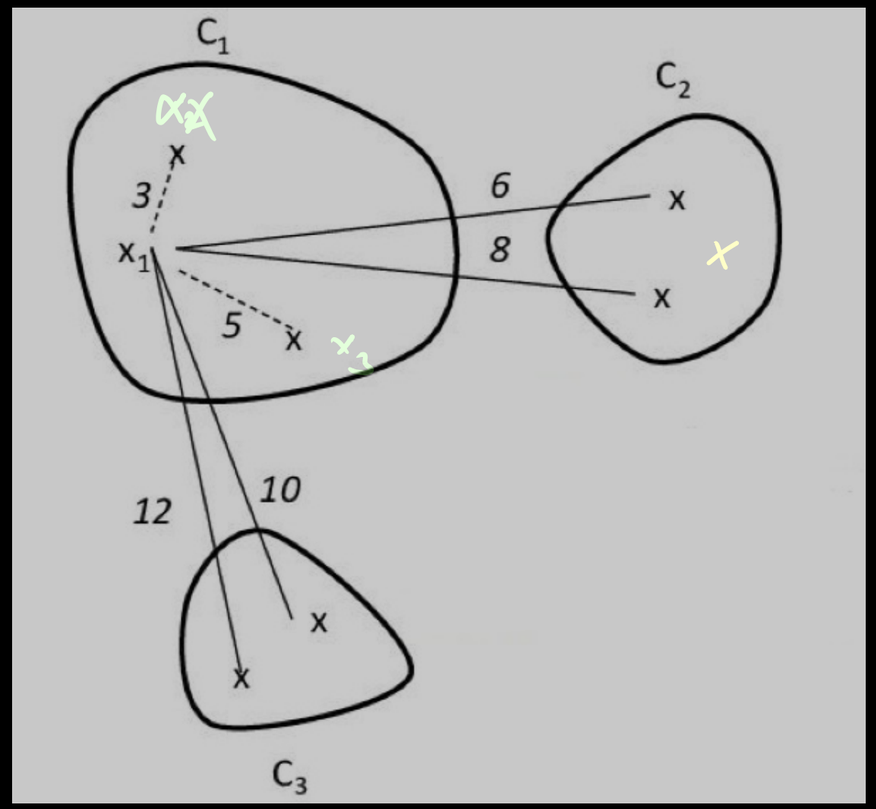
$$\text{Silhoute}(x_i) = \frac{(b-a)}{\max(b,a)}$$

a = Avg. within cluster distance from x_i

b = min (Avg. dist from x_i to other cluster)

a = Avg. within cluster distance from x_i

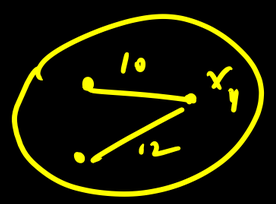
$$\text{Silhouette}(x_i) = \frac{(b-a)}{\max(b,a)}$$



x_1

$$a = \frac{3+5}{2} = 4$$

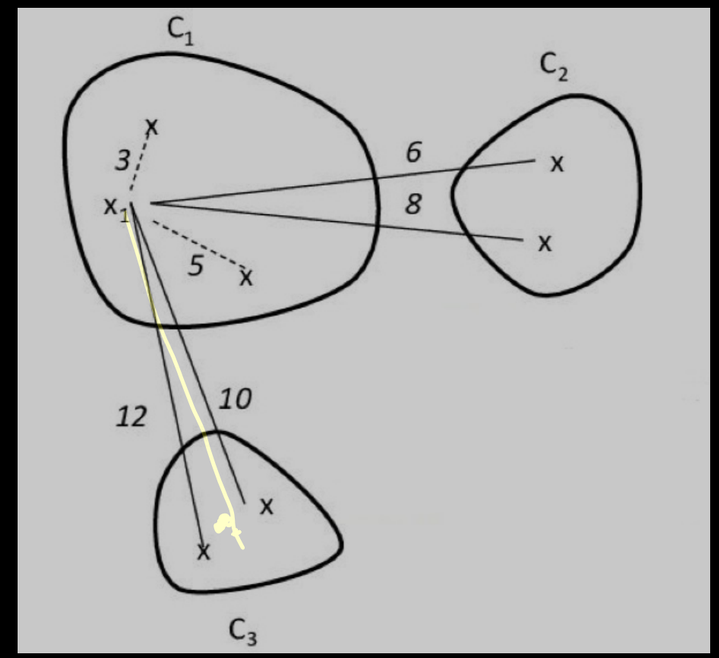
$$a = \frac{10+12}{2}$$



$b = \min(\text{Avg. dist from } x_i \text{ to other cluster})$

$$= \min\left(\underbrace{\frac{6+8}{2}}_{C_2}, \underbrace{\frac{12+10}{2}}_{C_3}\right)$$

$$= \min(7, 11) = 7$$



$$\frac{12+10+7}{3}$$

$$\text{Silhoute}(x_i) = \frac{(b-a)}{\max(b,a)}$$

$$= \frac{(7-4)}{\max(7,4)} = 3/7$$

Range of Silhouette Score

$$S_i(x_i) = \frac{b-a}{\max(b,a)} = \frac{b-0}{\max(b,0)}$$

$$= 1$$

$$\boxed{a=0}$$

$$b = 0$$

$$S_i(x_i) = \frac{0-a}{\max(0,a)} = -1$$

$$[-1, 1]$$

Time complexity of K-mean

m : Total datapoint

K : No of Cluster

i : No of iteration

d : dimension of datapoint

$$O(i * m * K \times d)$$

$i \to 200$, $K \to 10$

$$\approx O(m \times d)$$

$i < 200$

$K < 20$

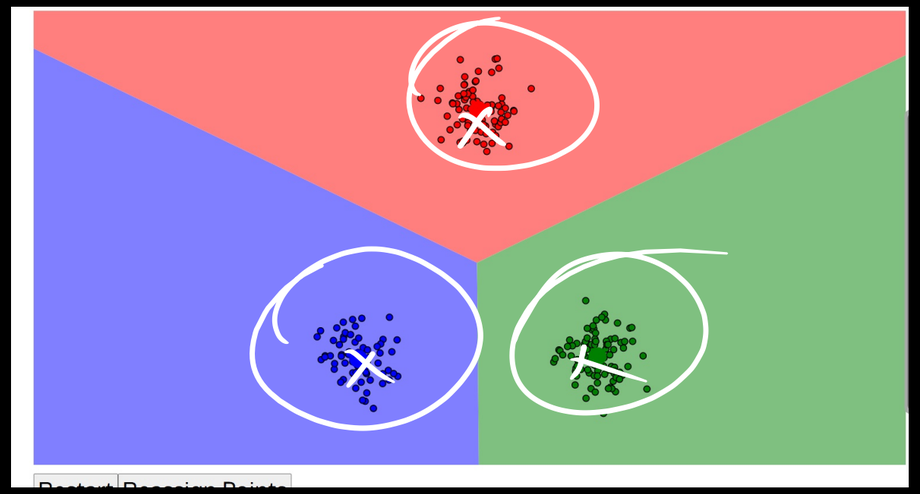
```python
# K-Means algorithm
def k_means(X, K, max_iters=100, tol=1e-4):
    centroids = initialize_centroids(X, K)
    for i in range(max_iters):
        clusters = assign_clusters(X, centroids)
        print(X.shape,clusters.shape)
        new_centroids = update_centroids(X, clusters, K)

        # Check for convergence
        if np.all(np.abs(new_centroids - centroids) < tol):
            break

        centroids = new_centroids
        display_results(X,clusters,centroids)
    return centroids, clusters
```

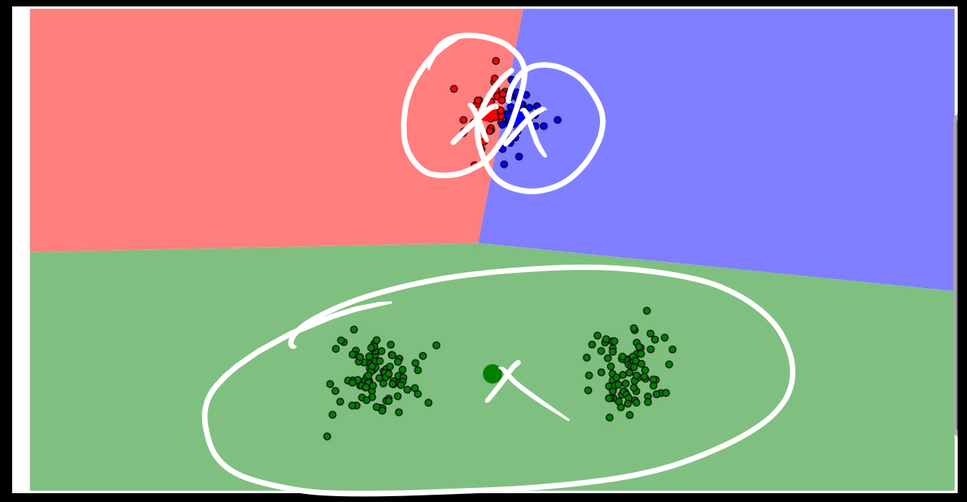
```python
# Function to assign points to the nearest centroid
def assign_clusters(X, centroids):

    distances = np.array([]).reshape(len(X),0)
    for centroid in centroids:
        distance = (((X - centroid)**2).sum(axis=1)**0.5).reshape(-1,1)
        distances = np.hstack([distances, distance])
    clusters = np.argmin(distances, axis=1)
    return clusters
```

```python
def update_centroids(X, clusters, K):
  new_centroids = []
  for k in range(K):
     centroid = X[clusters == k].mean(axis=0)
     new_centroids.append(centroid)
  return np.array(new_centroids)
```

→ Break until 22:17 PM

→ Issue of K-means





↳ 1) Initialization dependent

$K = 3$

(1) Run K-means multiple times
Random Initialization

$WCSS_1 \rightarrow 140.8$

$WCSS_2 \rightarrow 200$

$\vdots$

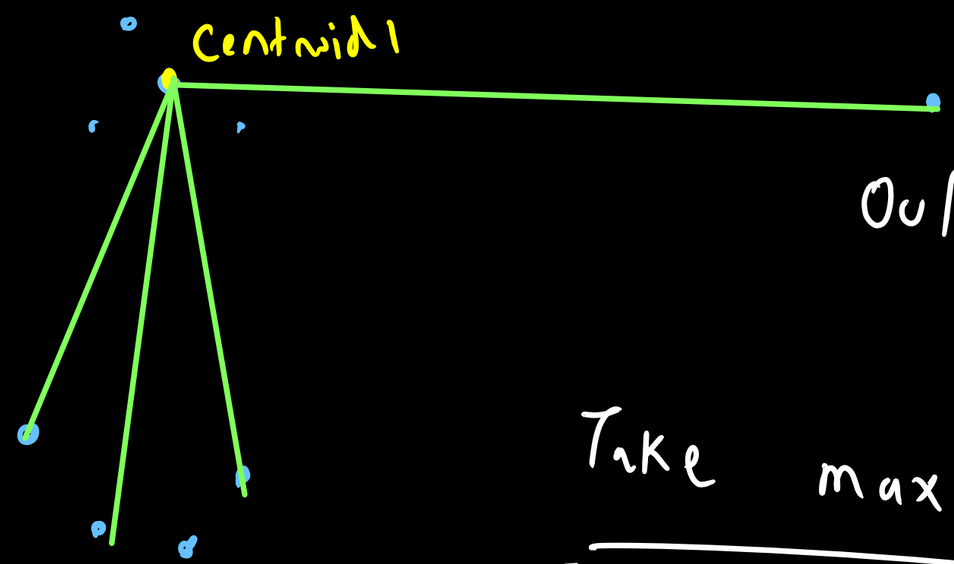
$WCSS_{10} \rightarrow 145$

$\rightarrow 140.8$

② k means ++

↳ Initialization strategy

K -centroid

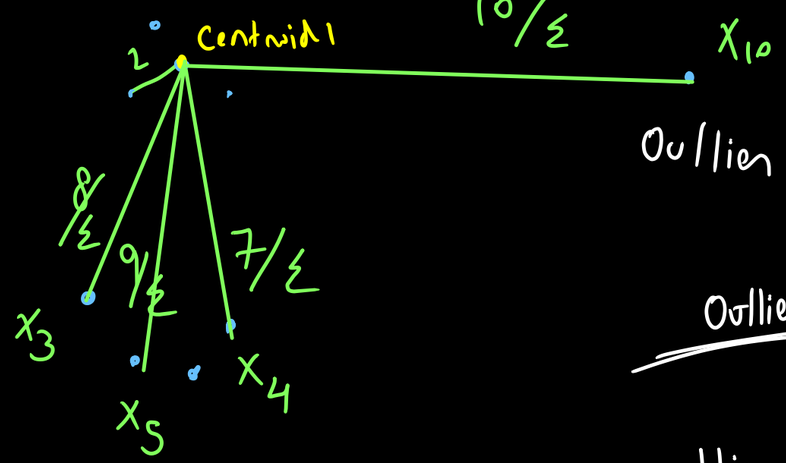


Outlier

Take max dist → Outlier

Run 10 times

- → Select 1st centroid randomly from any data point
- → Select 2nd centroid which is farther from 1st one $\boxed{P(\ ) \propto d}$
- → Run K-means and calculate WCSS



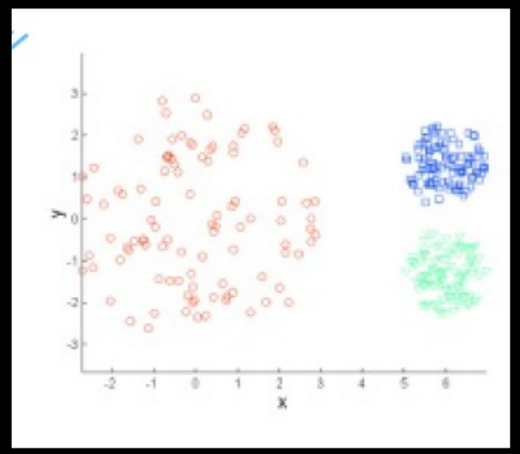
//

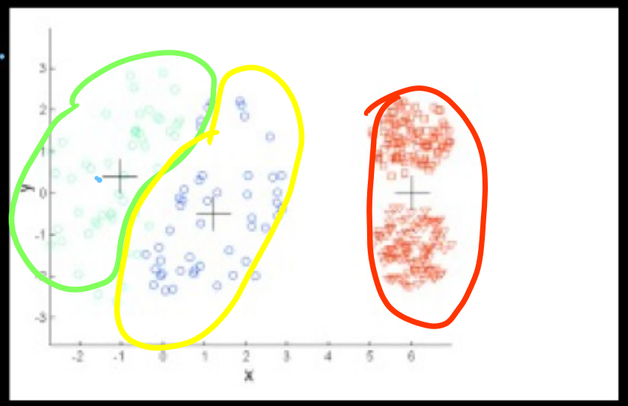
Run 10 times

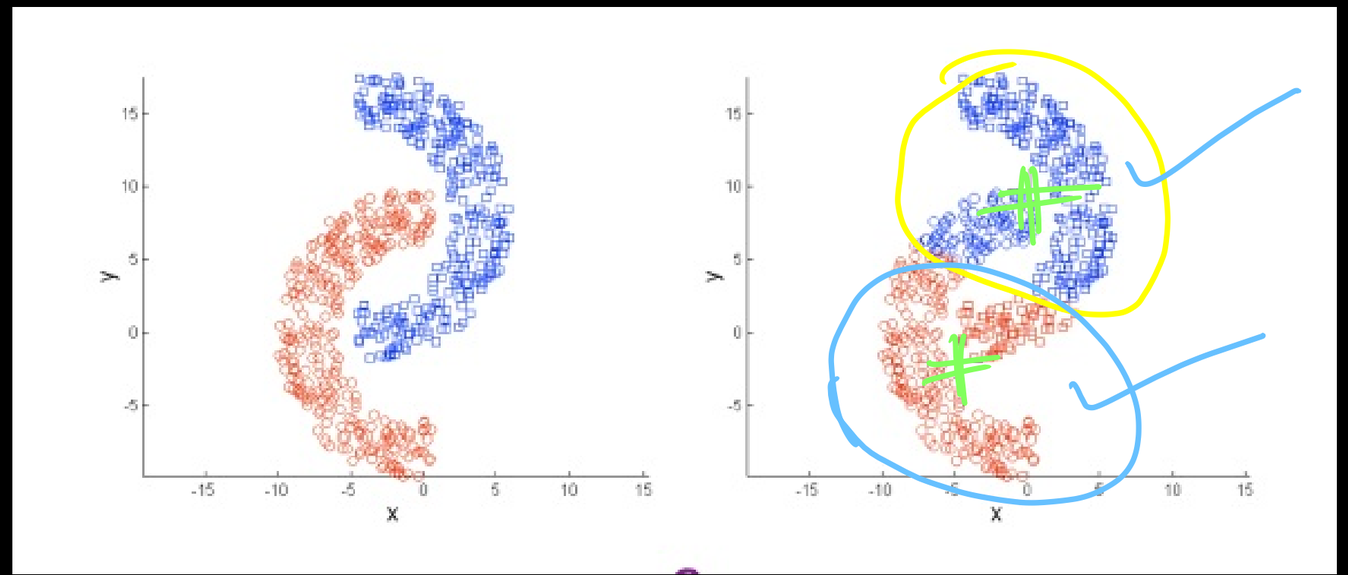
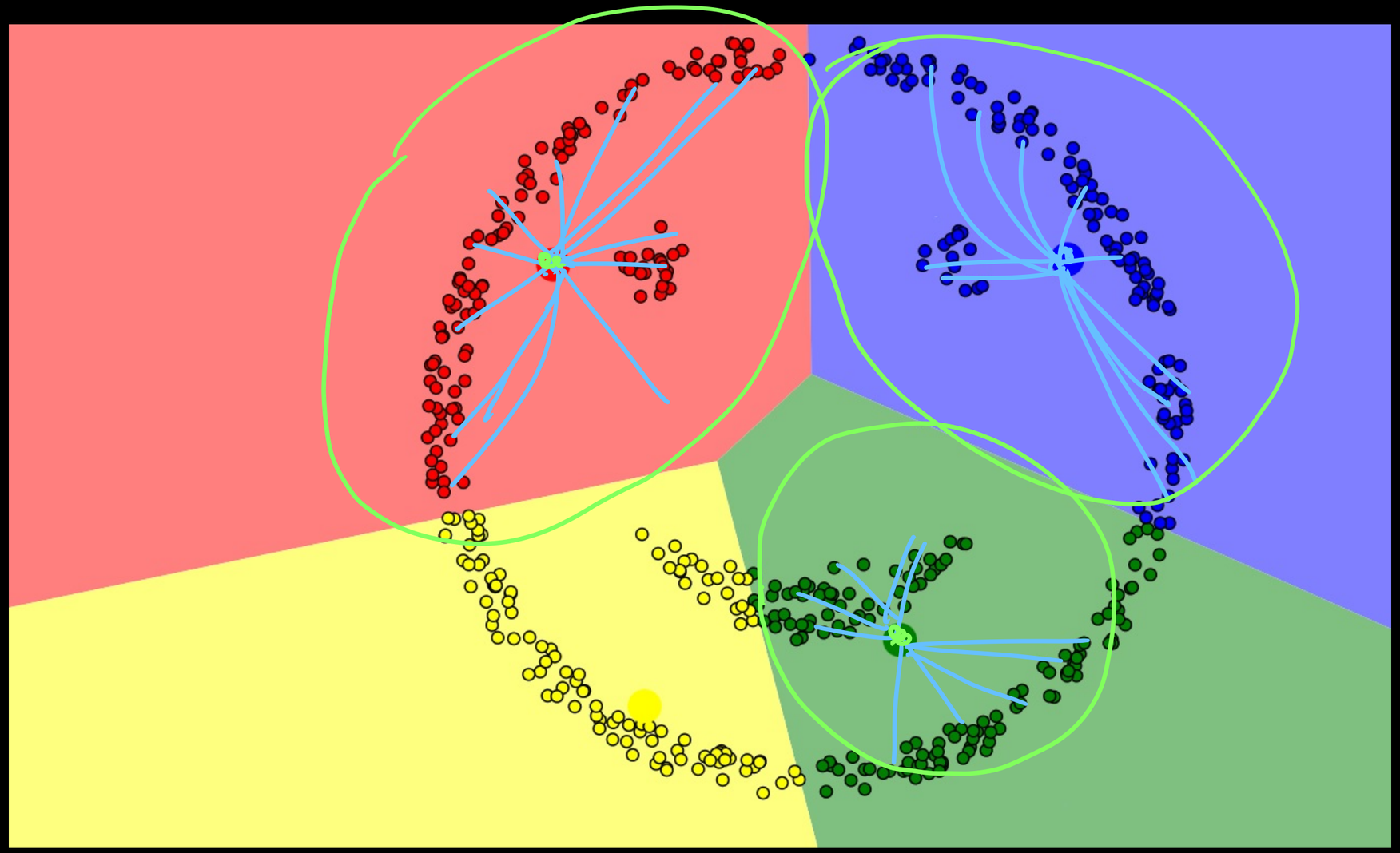
Outlier → x_{10} → 4

Non-outlier ← x_5 → 3, x_3 → 2, x_4 → 1

→ Different density





→ Non-globular structure

↳ K-means fails if clusters are non-spherical